SYSTEM ANALYSIS DOCUMENTATION (SAD)

BY TEAM BAKKUTTEH

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section introduces (Crypto-Integrated Financial Intelligence Advisor) CIFA as the strategic solution to Malaysia's fragmented financial data problem. It serves as a decision intelligence layer that unifies EPF (long-term investments), Fixed Deposit Rates (short-term investments), and crypto data (volatile investments) into a single, explainable workflow.

1.1. Purpose:

The system analysis document highlights the technical architecture and design-related decisions behind CIFA's development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *It's a local platform featuring a decoupled frontend-backend architecture. The React-based frontend and FastAPI-based backend operate as independent microservices, optimized for distributed deployment and high-performance data processing.*
2. ***Data Flows:***
    a. *The user's financial profile (income, expenses, and liabilities) flows from the React UI to the FastAPI backend, where it is enriched with live Bitcoin market data (via yfinance) and historical EPF/FD benchmarks. This data is then processed through a localized Python-based logic engine and an H5-based ML model before reaching the Z.AI GLM for deep financial reasoning and surplus allocation. Finally, the optimized strategic action plan is returned to the user's dashboard for real-time display.*
3. ***Model Process:***
    a. *The end-to-end workflow covers the following core features from: input of user's financial data, determining EPF status (way behind, behind or on track), calculation of surplus and AI reasoning for surplus allocation based on EPF, FD rates, and cryptocurrency.*
4. ***Role of Reference:***
    a. *This document provides a high-level technical reference for developers and judges to understand the system's robustness and scalability.*

*1.2. **Background**:*

Previously, financial planning was a manual, high-friction process requiring users to toggle between EPF i-Akaun, banking apps/websites for FD rates, and crypto exchanges. This led to making informed economic decisions complex and time-consuming, causing reckless and poor economic investments.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Customer	*Provide required financial data and understand the percentage-based surplus allocation*	*Clarity of surplus allocation, trust in reasoning and information accuracy*
Development Team	*Maintain the logic engine and GLM prompt pipeline.*	*Modular code, clear API contracts, and manageable token costs.*
QA Team	*Validating the system behavior for all the user roles*	*Defined scope, the request and response formats, and coverage of edge cases*
Z.AI (as Service)	*Process multi-step reasoning and strategy narratives.*	*Context-aware responses that respect local financial laws.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Responsive Web Dashboard (React)*
Architecture	*Distributed Logic Layer with Z.AI GLM as the "Reasoning Engine"*

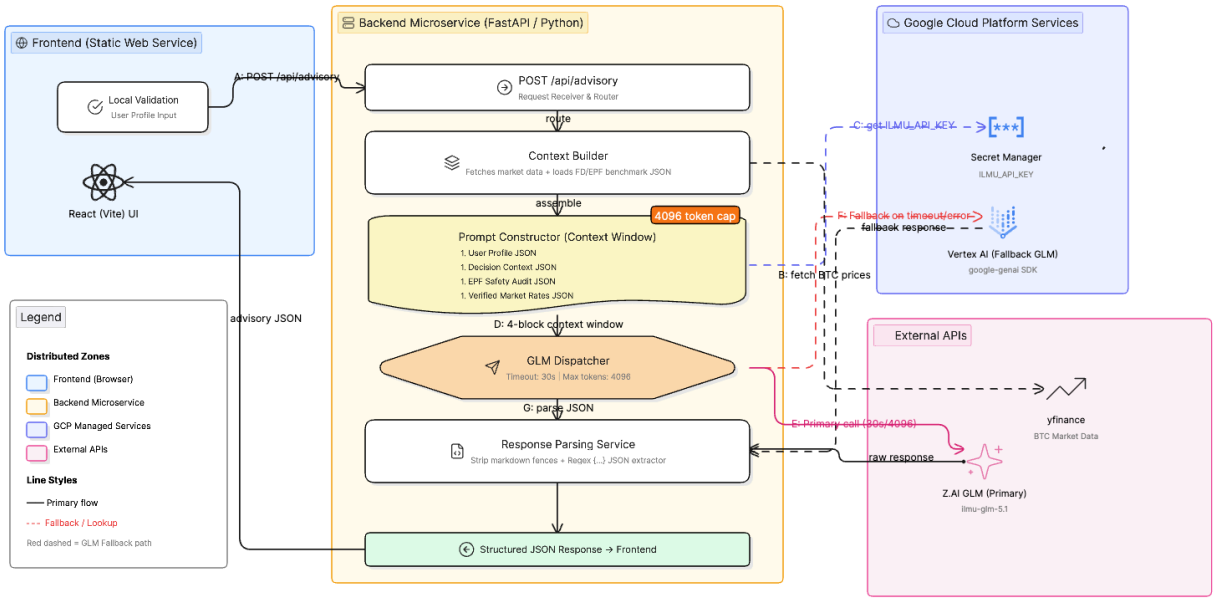
CIFA is structured as a modular, client-server system designed to ensure financial integrity while leveraging large language models for complex decision intelligence. The architecture consists of three primary layers: the frontend client (react) which is a responsive dashboard where users securely input data, backend layer (FastAPI) which performs calculations via python, and the intelligence layer (Z.AI GLM) which translates the raw numbers into percentage surplus allocations and produces reasonings.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

© UMHackathon 2026

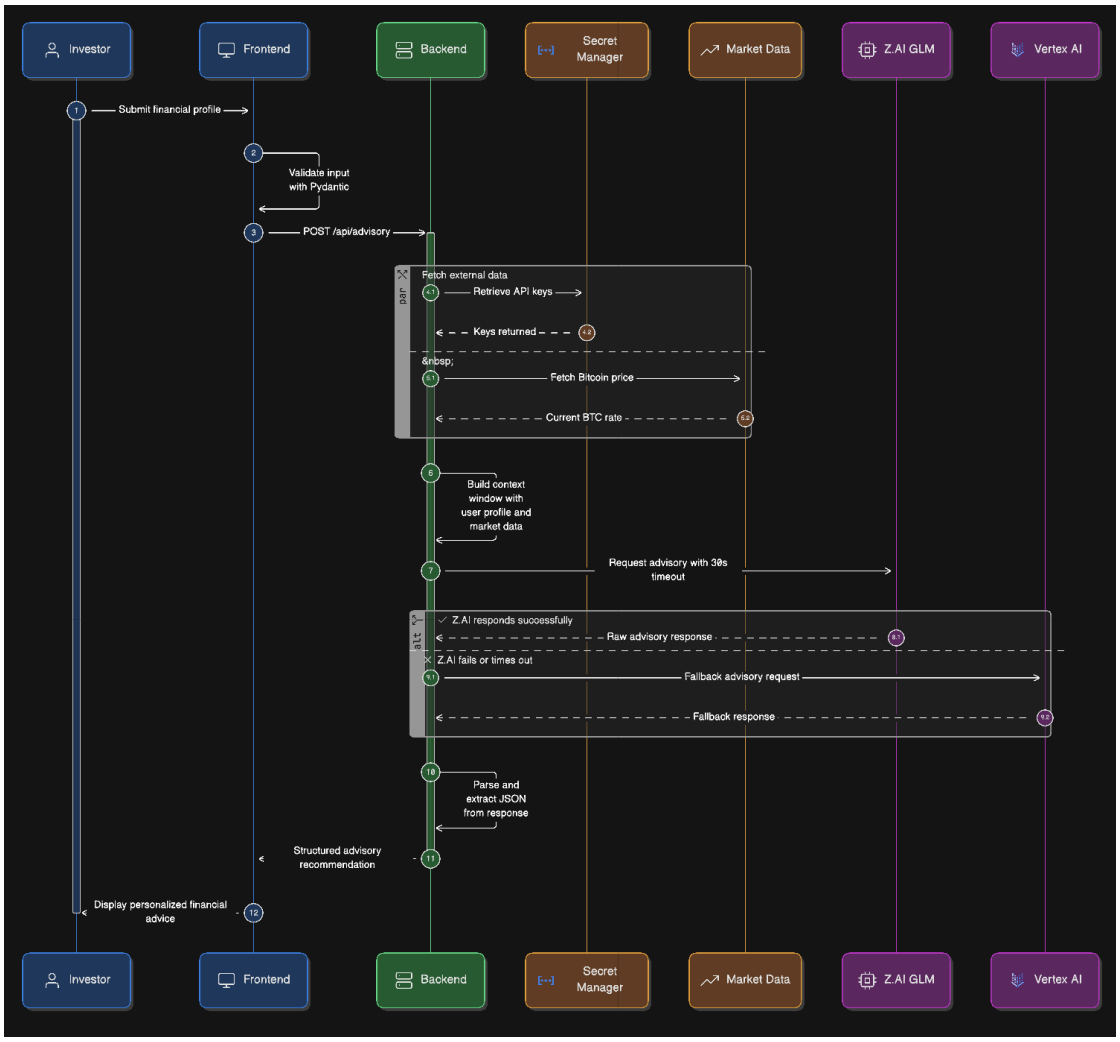
2.1.3 Dependency Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2 Sequence Diagram



2.2. Technological Stack

- *2.2.1. Frontend [React for a responsive, dashboard-style interface]*
- *2.2.2. Backend [Python (FastAPI) to handle the logic engine and Z.AI API integration]*
- *2.2.3. Database [None used, but a JSON file is used for user data]*
- *2.2.4. Cloud/Deployment [Google secret manager]*

2.3. Key Data Flows

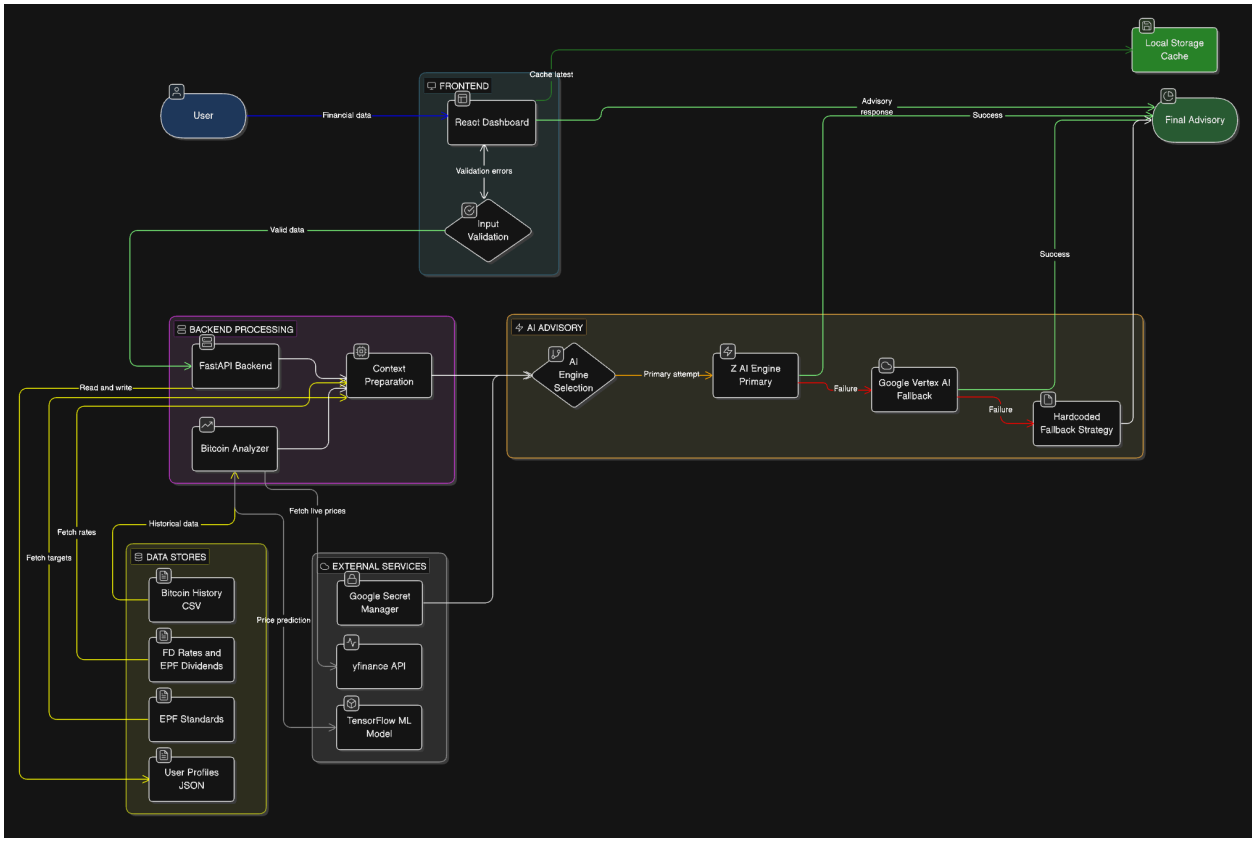
2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. *Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases*

b. *Show how proper laws are met (e.g. Black Hole)*

© UMHackathon 2026

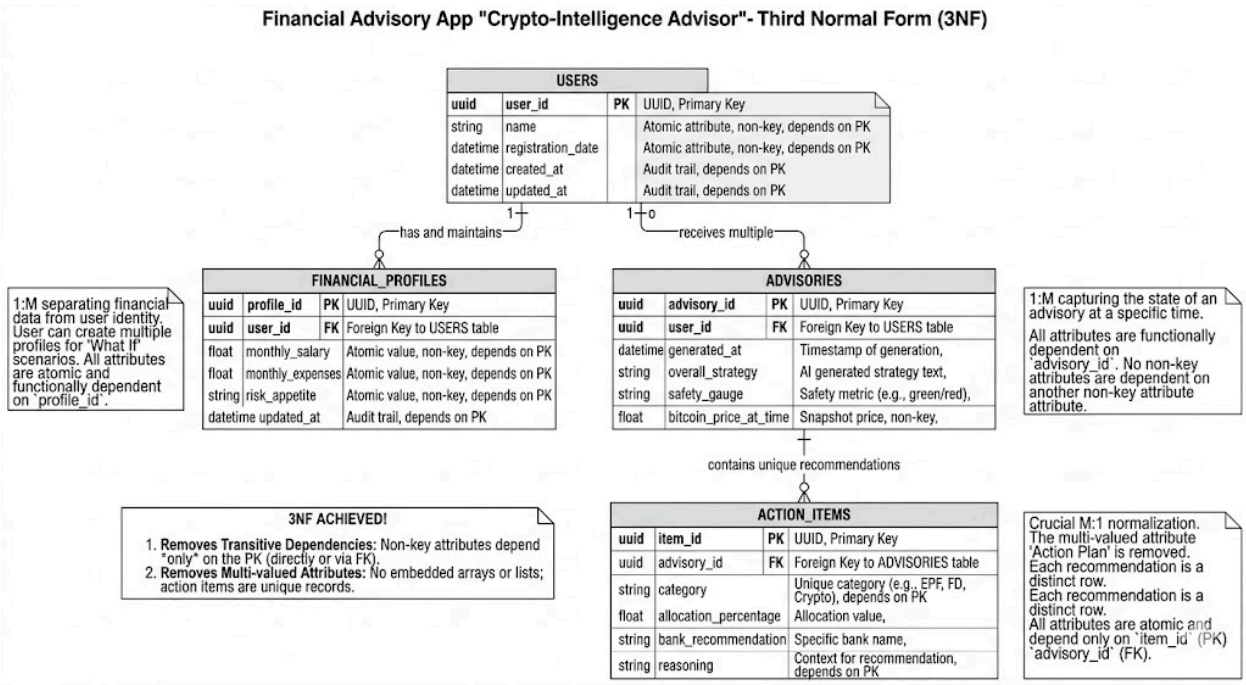


© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.



3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.*

#	*Features*	*Description*
1	*Real-Time Cryptocurrency*	*Integrates live Bitcoin rates to suggest "Buy/Hold/Sell" moments*
2	*Fixed Deposit Comparison*	*Ranks top Malaysian banks by rate and takes into account their minimum threshold (normally ranging between RM 500 and RM 1000)*

© UMHackathon 2026

3	*EPF Health Scoring*	*The user's current EPF balance is compared with the current EPF standard of their category (basic, adequate, and enhanced) and is decided on whether the user is currently "way behind", "behind" or "on track"*
4	*Conversational Reasoning*	*Z.AI explains how and why money should be allocated to the EPF, FD, and Crypto based on current market data and the unique user financial data*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	***Requirements***	***Implementation***
Scalability	*CIFA must handle sudden spikes in web traffic, particularly during peak Malaysian financial periods (e.g., EPF dividend announcement days or crypto bull runs).*	*Utilize AWS Auto Scaling Groups (ASG) for the FastAPI backend servers and an Elastic Load Balancer (ELB) to distribute incoming React web traffic smoothly.*
Reliability	*There should be 100% accuracy in the of the FD rates, live cryptocurrency market data, and EPF standards*	*EPF standards and FD rates are put into a JSON file and live crypto data is linked.*
Maintainability	*Updates to the financial logic, frontend UI, or AI prompts must be isolated to prevent system-wide crashes.*	*The React frontend and FastAPI backend are decoupled.*
Token Latency	*The API calls to the Z.AI GLM must return a response within 30s under normal load (p95) to prevent user drop-off.*	*Enforce a 30s hard timeout for asynchronous API calls. The React dashboard displays a skeleton loading state/spinner while awaiting the AI reasoning to maintain perceived performance.*
Cost Efficiency	*Average token cost per user session must not exceed the defined budget threshold (estimated at ~2,020 tokens)*	*System prompts (containing EPF rules) are cached for 24 hours. User inputs are truncated, and the AI's explanation output is*

© UMHackathon 2026

		capped at 150 words to prevent token bloating.
Robustness	*Handle API failures gracefully*	*If Z.AI GLM is down, reroute to Gemini AI. If all AI fails, serve the static Rule-Based mathematical results from the Python engine so the user still gets their EPF deficit numbers.*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a. *User Authentication, to implement secure login to protect the user's confidential data*
b. *Implementation of Stock Markets as an additional investment*
c. *Multi-Asset Portfolio Tracker, that tracks the user's current combined FD, EPF, and crypto holdings*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: New prompting logic or logic engine updates are initially released to 5% of active users. The system monitors the variance between the Python-calculated "Ground Truth" and the Z.AI GLM's narrative output before scaling to 100%. CIFA conducts A/B testing on narrative styles—comparing a* ***"Risk-Averse"*** *tone (prioritizing EPF security) against an* ***"Optimized Growth"*** *tone (prioritizing FD/Crypto surplus)—to determine which strategy yields higher user engagement and target completion.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: In the event of a "Logic Drift" (where AI reasoning contradicts the hard-coded math), an automated pipeline triggers an ER. This immediately redeploys the Golden Version, which restricts the UI to math-only results from the local logic engine, disabling the conversational AI layer until it is recalibrated.*

© UMHackathon 2026

4.1.3. Priority Matrix:

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project.

Priority	*Trigger Condition*	*Action*
P1: Critical	*Surplus allocation logic is incorrect*	*Immediate rollback to Golden Release; disable AI reasoning layer.*
P2: High	*Z.AI GLM latency exceeds 30 seconds at p95.*	*Automatically switch to Gemini 1.5 or use the default defined fallback strategy*
P3: Medium	*Live crypto data feeds fail to refresh for 60 minutes*	*Display cached data with a "Last Updated" warning banner.*
P4: Low	*Minor UI layout anomalies or non-critical styling bugs*	*No immediate deployment needed*

*4.2. **Monitoring**:*

*4.2.1. **Token Telemetry:** Real-time tracking of input/output token balance (target: ~2,020 tokens/session) to prevent budget overruns.*

*4.2.2. **Endpoint Latency:** Tracking the response time of the Z.AI API to ensure the "30s normal load" NFR is consistently met.*

***4.3. Assumptions:** State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

- ***a.** Users have access to their latest EPF i-Akuan balances for accurate user inputs*
- ***b.** Z.AI GLM API remains available with a consistent token pricing model*
- ***c.** The RM 500/1000 minimum barrier for Malaysian Fixed Deposits remains a standard market constraint for investment logic*

© UMHackathon 2026

4.4. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function.*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It is the core "decision intelligence" engine, which handles the multi-step reasoning, surplus allocation and reasoning. From which it produces a structured JSON output.*	*High - in the event that the GLM API fails, the website will be unable to fetch data and thus will fail. So, we have implemented a fallback strategy where Genimi will be used in Z.Ai's place. If Genimi does not work as well, a default rule-based response will be used.*
Gemini 1.5 API	*Acts as the primary failover reasoning layer if Z.AI fails.*	*Low - to ensure that consistent reasoning is provided, consistent prompt engineering templates are used.*
yfinance (API)	*Fetches real-time price feeds for Bitcoin (BTC-USD) to drive investment surplus logic.*	*Medium - Potential for IP rate limiting or service latency from Yahoo Finance. To mitigate this, we can implement a 5-minute cache for price data. Display a "Last Updated" timestamp to users if live data fails.*

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

Day 7: *Finalized program idea, and did job distribution*

Day 8: *Created mock data sets, sourced accurate financial and market data, prompt engineering, core logic, machine learning, backend, and frontend*

Day 9: *Frontend Integration, and program debugging*

Day 10*: Completion of QA, PRD, and SAD documentation*

© UMHackathon 2026

5.2. *Team Members Role:*

Kan Hoi Yeng *- Data Engineer - Source market and financial data*
Chew Jee Syuen *- Machine Learning Engineer - Prediction of future price movements for Bitcoin cryptocurrency*
Chong Wah Yun *- Intelligence Architect - Central reasoning engine that explains decisions in a "user-understandable manner"*
Ho Hann Yi *- Frontend Developer - UI/UX and explainability using React*
Lim Jie Xin *- Backend Developer - System Logic and optimization using Fast API*

5.3. *Recommendations:*

- ***LLM Context Optimization:*** *Utilize* ***Z.AI Prompt Caching*** *for the static EPF age-benchmark tables. Since these benchmarks only change annually, caching the system prompt can reduce input token costs by approximately 30–40% per session.*
- ***Microservices Architecture:*** *Transition the monolithic FastAPI backend into separate services for "Market Data" (Crypto/FD) and "User Analysis."*

© UMHackathon 2026